isec Engenharia

Design e Arquitectura de Software

Exame – Época Recurso

Ano Lectivo 2023-2024 Duração: 2 horas

1 – Considere um interface `IVideo` que representa um filme com uma determinada duração. Este interface especifica os seguintes métodos:

`int getLength()` – Devolve duração do vídeo em segundos.

`void play(Screen s, int startTime, PlayObserver obs)` – Toca o video no ecrã indicado, a partir do instante temporal indicado. A classe `Screen` representa um ecrã e os seus atributos e métodos não são relevantes. O objeto *obs* implementa o interface `PlayObserver` e é um observador do processo de reprodução. Assuma, para simplificação, que a notificação é gerada apenas quando o vídeo termina a sua reprodução.

`RGB getPixel(int x, int y, int time)` – Devolve o valor RGB (Red-Green-Blue) do pixel nas coordenadas (x,y) no instante time (tempo indicado em centésimos de segundo). A classe RGB disponibiliza os getters/setters necessários para aceder e modificar a informação da cor *(caso as coordenadas x,y estejam fora da área visível, deve devolver o valor RGB(0,0,0))*.

A classe `Video` é uma implementação do interface `IVideo`. Para além de implementar esse interface, implementa também o Interface `PlayObserver`.

Nas perguntas seguintes, pede-se para indicar como poderiam ser aplicados alguns padrões estudados para diferentes fins. Não se pretende aqui obter respostas que incluam descrições genéricas do funcionamento dos padrões em questão (i.e. palha)*, mas sim respostas que descrevam a aplicação concreta desses padrões no contexto indicado.*

a) [12.5%] Sugira qual ou quais os métodos que devem ser especificados no interface `PlayObserver` para que este possa ser um **observador** do processo de reprodução do vídeo. Indique em comentário qual a função desse (ou desses) método(s).

b) [12.5%] Considere que pretende-se suportar a edição de vídeo, permitindo a criação de um novo vídeo por composição de segmentos e extratos de outros vídeos. Por exemplo, pode ser criado um *video1* de 25 segundos que inclui os primeiros 10 segundos do *vídeoA*, seguidos dos últimos 10 segundos do *VideoB*, seguindo do conteúdo entre os 5 e 10s do *VideoC*. Para esse efeito, será utilizado o padrão **composto** (*composite*) para definir uma classe `VideoEditado` que representa o resultado da edição.
    a. Apresente um diagrama das classes e interfaces envolvidos, clarificando como é que estes se relacionam com a classe `Video` e interface `IVideo`.
    b. Apresente o código dos métodos da classe `VideoEditado`.

c) [12.5%] Pretende-se acrescentar a possibilidade de aplicar filtros aos vídeos. Os filtros modificam os pixéis que compõem as imagens do vídeo. Por exemplo, o filtro `RemoveRuido` vai fazer com que a cor do pixel na posição x,y seja igual à média das cores dos pixéis no quadrado entre as posições (x-1,y-1) e (x+1,y+1). A média é feita componente a componente do valor RGB correspondente. Por exemplo, a média das cores RGB (100,150,200) e (0,150,100) será (50,150,150). Para esse efeito, pretende-se usar o padrão **decorador**.

a. Apresente um diagrama das classes e interfaces envolvidos, clarificando como é que estes se relacionam com a classe `Video` e interface `IVideo`.
b. Apresente o código dos métodos da classe `RemoveRuido`.

d) [12.5%] Pretende-se criar uma classe `VideoBuilder` que é um **builder** que pode ser usado para criar vídeos editados e filtrados.
   a. Apresente um diagrama das classes e interfaces envolvidos, clarificando como é que estes se relacionam com a classe `Video` e interface `IVideo`.
   b. Apresente o código dos métodos da classe `VideoBuilder`.
   c. Indique como é que esta classe pode ser utilizada para criar um novo vídeo que inclui:
      i. Os primeiros 10 segundos do vídeo *introdução*.
      ii. O segmento entre os 5 e os 15 segundos do vídeo *reportagem*, filtrado com o filtro `RemoveRuido`.
      iii. O vídeo *créditos*, na sua totalidade.

2- Pretende-se possibilitar, através de um WebAPI adequado, a edição de vídeos.

Para esse efeito, são disponibilizados dois recursos: **Video** (que representa um vídeo) e **EditorVideo** (que representa um processo de edição de um vídeo – com funcionalidade similar ao builder descrito na alínea 1 d)).

Os recursos associados a Videos incluem campos com o nome do vídeo e possivelmente o ficheiro binário correspondente. Disponibilizam também métodos custom para fazer upload/download do respetivo ficheiro.

Os recursos EditorVideo disponibilizam métodos custom que permitem adicionar um vídeo e/ou aplicar um filtro a um segmento de vídeo pertencente a essa edição. Alguns exemplos de utilização seriam:

- `POST /videos/ {name:novo}`
  Cria um vídeo com nome "novo".

- `POST /videos/novo:upload?name=1.mpeg`
  Faz o upload de um ficheiro binário para o vídeo "novo". Este ficheiro terá o nome 1.mpeg.

- `POST /videos/novo:download`
  Faz o download do ficheiro binário do vídeo "novo"

- `POST /editoresVideo/editor1:addVideo?name=intro?source=video1`
  Acrescenta o *video1* como um novo segmento do conteúdo gerado pelo *editor1*. Este novo segmento tem o nome *intro*.

- `/editoresVideo/editor1/intro:addFilter?name=RemoveRuido`
  Acrescenta o filtro *remove ruido* ao segmento *intro* do vídeo a ser criado com o editor 1.

- `/editoresVideo/editor1:generateVideo?name=VideoNovo`
  Gera um novo recurso que contém o vídeo gerado pelo editor 1. O novo recurso é designado como "VideoNovo".

2.1 [20%] Um problema que se observou é que os utilizadores frequentemente chamam o método custom *generateVideo* seguido imediatamente de uma chamada ao método *download* do recurso vídeo criado, para obter o ficheiro vídeo correspondente. No entanto, a geração do ficheiro binário do vídeo é um processo demorado, pelo que nestas situações o API bloqueia no método download a aguardar que o ficheiro seja gerado, originando uma má experiência de utilização. Pretendemos resolver esta situação:

a) Indique qual o padrão estudado que poderia ser utilizado nesta circunstância (basta indicar o nome).
b) Indique através de um diagrama de sequência como é que seria usado nesta situação concreta (Chamada da função generateVideo seguida de uma chamada imediata ao método download do serviço criado).

2.2 [20%] Pretende-se simplificar o processo de criação de vídeos de noticias. O processo é sempre similar: o vídeo começa com o segmento de introdução (*intro*), seguido de um vídeo com a notícia, seguido de um segmento de fecho (*outro*). O vídeo da notícia deve ter aplicado o filtro *ReduçãoRuido*. Isto obriga à realização de um conjunto repetitivo de chamadas à API sempre que se quer criar um destes vídeos. Pretende-se simplificar o processo, permitindo que sejam criadas novas noticias, de forma simplificada, da seguinte maneira:

```
POST /tarefas/crianoticia:execute?name=noticia1?videonoticia=news1
```

Esta chamada ao API indica que deve ser criada uma nova noticia com o nome *noticia1* e vídeo de noticias *news1*. A *intro* e *outro* e filtragem são aplicadas automaticamente conforme descrito atrás. Dá-se também início à preparação do respetivo ficheiro binário do vídeo para download.

a) Indique qual o padrão estudado que está a ser aplicado nesta circunstância (basta indicar o nome).
b) Indique através de um diagrama de sequência como é que o cliente poderia obter o ficheiro binário com o vídeo gerado. Comente as várias chamadas ao API se achar necessário fornecer algum esclarecimento.

2.3 [10%] Alguém na equipa de desenvolvimento lembrou-se que um recurso **Vídeo** que está a ser utilizado num **EditorVideo** pode entretanto ser apagado, e levantou a questão sobre qual a melhor forma de lidar com o processo. Indique duas alternativas possíveis, indicando as vantagens e desvantagens de cada uma delas.